

Bachelorthesis
Attribute Stream-Based Access Control im
Linux-Kernel

Christian Koch

15. Juni 2026

Inhaltsverzeichnis

1	Einleitung	4
1.1	Motivation	4
1.2	Wissenschaftliche Fragestellung	4
1.3	Zielsetzung	4
1.4	Aufbau der Arbeit	4
2	Grundlagen	5
2.1	Zugriffskontrolle und Reference-Monitor-Konzept	5
2.2	Attribute-Based Access Control	6
2.3	Attribute Stream-Based Access Control	6
2.4	Policy-Beschreibungen für ASBAC	7
2.5	Userspace und Kernel-space in Linux	7
2.6	Linux Security Modules und LSM-Hooks	8
2.7	eBPF und eBPF-LSM	8
2.8	eBPF-Maps und gepinnte Maps	9
2.9	eBPF-Verifier, Stack und Programmkomplexität	9
2.10	Aya als Rust-Framework für eBPF	10
2.11	Dynamische Attribute und Policy-Generatoren	11
2.12	Überwachung bestehender Zugriffe und File Descriptors	11
3	Anforderungsanalyse	13
3.1	Funktionale Anforderungen	13
3.2	Operative Anforderungen	15
3.3	Entwicklungsbezogene Anforderungen	15
4	Konzeption	16
4.1	praktische Einschränkungen von eBPF	16
4.2	Architektur	16
4.3	Datenfluss	17
4.3.1	Policy-Datenfluss	17
4.3.2	Datenfluss dynamischer Attribute	18
4.3.3	Zugriffsentscheidungen im Kernel	18
4.3.4	Monitoring bestehender Zugriffe	18
4.4	Komponenten	19
4.4.1	Userspace-Komponenten	19
4.4.2	Kernel-space-Komponenten	20
4.4.3	Administrationskomponente	20
4.5	Datenobjekte	20
4.5.1	Datenobjekte im Userspace	20
4.5.2	Datenobjekte im Kernel-space	21
4.6	Entwurfsentscheidungen	21
4.6.1	Trennung von Userspace und Kernel-space	21
4.6.2	Einsatz von eBPF-LSM statt klassischem Kernelmodul	22
4.6.3	Auswahl der LSM-Hooks	23
4.6.4	Policy-Verarbeitung im Userspace	23

4.6.5	Reduzierte Policy-Repräsentation im Kernel-space	23
4.6.6	Kommunikation über eBPF-Maps	23
4.6.7	Generationenmodell für Policy-Aktualisierungen	23
4.6.8	Trennung statischer Policies und Stream-policies	23
4.6.9	Aufteilung der Policy-Maps nach Hook und Policy-Typ	23
4.6.10	Tail-Call-basierte Aufteilung der eBPF-Programme	23
4.6.11	Combining-Strategie für Policy-Entscheidungen	23
4.6.12	Identifikation von Dateien über Device und Inode	23
4.6.13	Nachträgliche Überwachung durch den Monitor	23
4.6.14	Enforcement durch Schließen von File Descriptors	23
4.6.15	Administration und Diagnose über ein separates Tool	23
5	Entwurf und Umsetzung	24
5.1	Komponente 1	24
5.2	Komponente 2	24
5.3	Komponente 3	24
5.4	Implementierungsdetails	24
6	Evaluation	25
6.1	Testumgebung	25
6.2	Testszenarien	25
6.3	Ergebnisse	25
7	Zusammenfassung und Ausblick	26

1 Einleitung

1.1 Motivation

Die Sicherheit von Linux-Systemen ist ein fundamentales Anliegen in der heutigen IT-Landschaft. Das Linux Security Module Framework stellt Mechanismen für verschiedene Sicherheitsprüfungen mittels Attribute-Based Access Control im Linux-Kernel dar[16]. LSM-Hooks ermöglichen es, sicherheitsrelevante Kerneloperationen abzufangen und auf dieser Grundlage Zugriffsentscheidungen zu treffen. Ein LSM-Hook prüft einen Zugriff jedoch grundsätzlich zu dem Zeitpunkt, an dem die jeweilige Kerneloperation ausgeführt wird. Bei dauerhaft bestehenden Zugriffen wird eine spätere Änderung der Entscheidungsgrundlage dadurch nicht automatisch erneut bewertet.

1.2 Wissenschaftliche Fragestellung

Diese Arbeit untersucht, inwieweit Attribute Stream-Based Access Control im Linux-Kernel prototypisch umgesetzt werden kann. Im Mittelpunkt steht dabei die Frage, wie dynamische Attribute aus dem Userspace für Zugriffsentscheidungen im Kernel bereitgestellt und bei Änderungen nachträglich berücksichtigt werden können.

Daraus ergibt sich folgende wissenschaftliche Fragestellung:

Wie kann Attribute Stream-Based Access Control mit dynamischen Attributen im Linux-Kernel prototypisch umgesetzt werden, und welche technischen Einschränkungen ergeben sich dabei?

Zur Beantwortung dieser Fragestellung werden insbesondere die Überführung textueller Policy-Dateien in eine kernelgeeignete Repräsentation, die Bereitstellung dynamischer Attribute für Zugriffsentscheidungen im Kernel sowie die nachträgliche Überwachung bestehender Zugriffssituationen betrachtet.

1.3 Zielsetzung

Zielsetzung der Arbeit ist ein Prototyp, anhand dessen eine Zugriffskontrolle mittels Attribute Stream-Based Access Control im Linux-Kernel stattfindet. Insbesondere sollen Stream-Attribute wie Zeitinformationen sowie externe, zur Laufzeit aktualisierte Attribute in die Zugriffsentscheidung einbezogen werden. Ziel ist die Implementierung eines Prototyps, der Policies einliest, in den Kernel überträgt und Zugriffsentscheidungen anhand dieser Policies trifft. Darüber hinaus soll der Prototyp bereits bestehende Zugriffssituationen bei nachträglich geänderten Bedingungen erkennen und einschränken können.

1.4 Aufbau der Arbeit

2 Grundlagen

Dieses Kapitel beschreibt die fachlichen und technischen Grundlagen, auf denen der in dieser Arbeit entwickelte Prototyp aufbaut. Zunächst werden grundlegende Konzepte der Zugriffskontrolle eingeordnet, insbesondere das Reference-Monitor-Konzept sowie attributbasierte Zugriffskontrollmodelle. Darauf aufbauend wird erläutert, wie Attribute in dynamischen Systemen als veränderliche Entscheidungsgrundlage verwendet werden können und welche Rolle textuelle Policy-Beschreibungen dabei einnehmen.

Für die Umsetzung im Linux-Kernel sind außerdem die Trennung zwischen Userspace und Kernelspace, Linux Security Modules sowie eBPF relevant. Diese Mechanismen bestimmen, an welchen Stellen Zugriffsentscheidungen technisch getroffen werden können und welche Einschränkungen bei der Implementierung im Kernel gelten. Ein besonderer Fokus liegt dabei auf eBPF-Maps als Schnittstelle zwischen Userspace und Kernelspace, gepinnten Maps für dauerhaft zugreifbare Zustände sowie den Vorgaben des eBPF-Verifiers.

Abschließend werden die für den Prototyp spezifischen Grundlagen betrachtet: die Verwendung von Aya als Rust-Framework, die Abbildung dynamischer Attribute und Policy-Generationen sowie die Überwachung bereits bestehender Zugriffe über File Descriptors. Damit schafft das Kapitel die Grundlage für die anschließende Anforderungsanalyse und die spätere Architektur des Systems.

2.1 Zugriffskontrolle und Reference-Monitor-Konzept

Zugriffskontrolle beschreibt Mechanismen, mit denen entschieden wird, ob ein Subjekt eine bestimmte Operation auf einer Ressource ausführen darf. Ein Subjekt kann dabei beispielsweise ein Prozess oder ein Benutzer sein, während Ressourcen Dateien, Netzwerkendpunkte oder andere geschützte Objekte des Systems sein können. Eine Zugriffsentscheidung basiert auf einer Menge von Regeln oder Policies, die festlegen, unter welchen Bedingungen eine Operation erlaubt oder verweigert wird. Diese Sichtweise knüpft an grundlegende Begriffe der Informationsschutzmechanismen an, bei denen Akteure, geschützte Objekte und Berechtigungen als zentrale Elemente der Zugriffskontrolle betrachtet werden [10].

Ein grundlegendes Konzept für die Durchsetzung von Zugriffskontrolle ist der Reference Monitor. Der Reference Monitor ist eine abstrakte Sicherheitskomponente, die jeden Zugriff eines Subjekts auf ein geschütztes Objekt überprüft, bevor dieser Zugriff ausgeführt wird. Damit ein Reference Monitor wirksam ist, muss er insbesondere drei Eigenschaften erfüllen: Er muss bei jedem relevanten Zugriff durchlaufen werden, gegen Manipulation geschützt sein und hinreichend klein beziehungsweise nachvollziehbar sein, damit seine Korrektheit überprüft werden kann [1].

Für Betriebssysteme ist dieses Konzept besonders relevant, da Zugriffe auf Dateien, Prozesse oder andere Systemressourcen letztlich über den Kernel vermittelt werden. Wird Zugriffskontrolle erst außerhalb des Kernels umgesetzt, besteht die Gefahr, dass sicherheitsrelevante Operationen nicht vollständig erfasst oder durch alternative Zugriffspfade umgangen werden. Eine Zugriffskontrolle im Kernel kann dagegen an zentralen Stellen ansetzen, an denen Operationen auf geschützte Ressourcen tatsächlich ausgeführt werden. Dies steht im Zusammenhang mit dem Prinzip der vollständigen Vermittlung, nach dem

jeder Zugriff auf ein geschütztes Objekt vor seiner Ausführung geprüft werden muss [10].

Im Kontext dieser Arbeit bildet das Reference-Monitor-Konzept die Grundlage für die Frage, wie Attribute Stream-Based Access Control im Linux-Kernel umgesetzt werden kann. Der Prototyp soll Zugriffe nicht nur anhand statischer Regeln bewerten, sondern dynamische Attribute in die Entscheidung einbeziehen. Dadurch entsteht zusätzlich die Herausforderung, dass eine zunächst erlaubte Zugriffssituation später unzulässig werden kann, wenn sich entscheidungsrelevante Attribute ändern. Das klassische Reference-Monitor-Konzept erklärt daher zunächst die unmittelbare Kontrolle eines Zugriffs, während die spätere Arbeit zusätzlich betrachtet, wie bestehende Zugriffssituationen nachträglich überwacht werden können.

2.2 Attribute-Based Access Control

Attribute-Based Access Control (ABAC) ist ein Zugriffskontrollmodell, bei dem eine Autorisierungsentscheidung nicht allein anhand einer festen Subjektidentität, wie bei ACL- oder identitätsbasierter Zugriffskontrolle, oder anhand einer zugewiesenen Rolle, wie bei Role-Based Access Control (RBAC), getroffen wird. Stattdessen werden Attribute ausgewertet, die sich auf das Subjekt, die Ressource, die angeforderte Operation und gegebenenfalls auf die Umgebung beziehen. Das National Institute of Standards and Technology (NIST) beschreibt ABAC als logische Zugriffskontrollmethode, bei der solche Attribute gegen Policies, Regeln oder Beziehungen geprüft werden, um zu entscheiden, ob eine Operation zulässig ist [4].

Für diese Arbeit ist ABAC relevant, weil dadurch Zugriffsentscheidungen feiner modelliert werden können als bei rein rollenbasierten oder benutzerbasierten Ansätzen. Ein Subjekt kann beispielsweise über Attribute wie Position, Benutzerkennung oder Gruppenzugehörigkeit beschrieben werden. Eine Ressource kann Attribute wie Pfad, Typ oder Schutzstufe besitzen. Zusätzlich können Umwelt- oder Systemattribute, etwa Uhrzeit oder Systemzustand, in die Entscheidung einbezogen werden. Damit entsteht ein flexibles Modell, in dem Policies Bedingungen über mehrere Attributklassen ausdrücken können.

ABAC trennt damit die Beschreibung der Entscheidungsgrundlage von der konkreten technischen Durchsetzung. Die Policy beschreibt, unter welchen Bedingungen ein Zugriff erlaubt oder verweigert werden soll. Die Auswertung dieser Bedingungen und die Durchsetzung der Entscheidung können dagegen von unterschiedlichen Systemkomponenten übernommen werden. Diese Trennung ist für den Prototyp wichtig, weil die Policy-Verarbeitung im Userspace stattfinden kann, während die Zugriffsdurchsetzung später im Kernel betrachtet wird.

2.3 Attribute Stream-Based Access Control

Attribute Stream-Based Access Control (ASBAC) wird in dieser Arbeit als Erweiterung von ABAC verstanden, bei der Attribute nicht nur als statische Werte betrachtet werden. Stattdessen können Attribute zur Laufzeit aktualisiert werden und dadurch einen Strom von Zustandsänderungen bilden. Eine Zugriffsentscheidung hängt dann nicht nur von den Attributwerten zum Zeitpunkt der ersten Prüfung ab, sondern auch davon, ob sich diese Werte während einer bestehenden Zugriffssituation ändern.

Diese Sichtweise steht inhaltlich in der Nähe von Usage Control. Unter Usage Control

werden Zugriffskontrollmodelle verstanden, die eine Nutzung nicht nur zum Zeitpunkt der initialen Autorisierung betrachten, sondern auch Bedingungen während der laufenden Nutzung einbeziehen. Usage Control erweitert klassische Zugriffskontrolle unter anderem um die Veränderbarkeit von Attributen und die Fortdauer von Entscheidungen während einer Nutzung. Gouglidis et al. beschreiben diese Konzepte als *attribute mutability* und *continuity of decision* [3]. Für ASBAC bedeutet dies, dass ein zunächst erlaubter Zugriff später erneut relevant werden kann, wenn sich ein entscheidungsrelevantes Attribut ändert.

Im Kontext dieser Arbeit ist dieser Punkt zentral. Ein Zugriff auf eine Datei kann beim Öffnen zunächst zulässig sein, weil die aktuellen Attribute die Policy erfüllen. Ändert sich später beispielsweise ein Subjektattribut oder ein Systemattribut, kann dieselbe Zugriffssituation nach der aktuellen Policy-Lage unzulässig werden. ASBAC beschreibt damit den fachlichen Bedarf, Attributänderungen nicht nur für zukünftige Zugriffe, sondern auch für bereits bestehende Zugriffssituationen zu berücksichtigen.

2.4 Policy-Beschreibungen für ASBAC

Policies beschreiben die Bedingungen, unter denen eine Zugriffsentscheidung getroffen wird. In einem attributbasierten Modell beziehen sich diese Bedingungen auf Attribute von Subjekt, Ressource, Aktion und Umgebung. NIST beschreibt Policies in ABAC als Regeln oder Beziehungen, gegen die Attribute ausgewertet werden [4]. Für ASBAC bleibt diese Grundidee erhalten, sie wird jedoch um dynamisch veränderliche Attribute erweitert.

Eine Policy-Beschreibung muss dafür ausdrücken können, welche Aktion auf welche Ressource bezogen ist und welche Attributbedingungen erfüllt sein müssen. Im Prototyp kann dies beispielsweise bedeuten, dass das Öffnen einer bestimmten Datei nur erlaubt ist, wenn ein Subjektattribut einen bestimmten Wert besitzt. Die Policy-Datei bleibt dabei eine textuelle Beschreibung im Userspace. Für die Auswertung im Kernel muss diese Beschreibung später in eine kernelgeeignete Repräsentation überführt werden.

Etablierte Zugriffskontrollarchitekturen unterscheiden häufig zwischen Komponenten zur Entscheidung und Komponenten zur Durchsetzung. XACML beschreibt beispielsweise eine Architektur mit Policy Decision Point und Policy Enforcement Point sowie eine Policy-Sprache zur Beschreibung attributbasierter Entscheidungen [8]. Diese Arbeit übernimmt nicht XACML als Sprache, nutzt aber dieselbe grundsätzliche Trennung: Policies werden außerhalb des Durchsetzungspunktes verwaltet und in eine Form überführt, die zur technischen Zugriffskontrolle verwendet werden kann.

2.5 Userspace und Kernelspace in Linux

Linux trennt zwischen Userspace und Kernelspace. Anwendungen laufen im Userspace und greifen auf Betriebssystemfunktionen über Systemaufrufe oder andere definierte Schnittstellen zu. Der Kernelspace enthält dagegen den Kernel und damit Code, der privilegiert ausgeführt wird und zentrale Ressourcen wie Prozesse, Speicher, Dateien und Geräte verwaltet. Die Kernel-Dokumentation beschreibt unterschiedliche Ausführungskontexte, darunter Code, der einem Prozess im Kernelspace zugeordnet ist, sowie Prozesse, die im Userspace laufen [9].

Diese Trennung ist für Sicherheitsmechanismen wesentlich. Komplexe Verarbeitung, Dateiparsing und Validierung sind im Userspace leichter zu entwickeln, zu testen und bei

Fehlern weniger kritisch für die Stabilität des Gesamtsystems. Code im Kernelspace hat dagegen unmittelbaren Einfluss auf das Betriebssystem. Fehler in Kernelcode können nicht nur eine Anwendung, sondern das gesamte System beeinträchtigen. Daher ist es sinnvoll, umfangreiche Policy-Verarbeitung im Userspace zu belassen und den Kernelspace auf die sicherheitskritische Durchsetzung vorbereiteter Entscheidungen zu beschränken.

Für diese Arbeit ergibt sich daraus eine Architekturgrenze. Der Userspace liest Policies und Attribute ein, validiert sie und übersetzt sie in eine Form, die im Kernel effizient ausgewertet werden kann. Der Kernelspace übernimmt dagegen die Entscheidung an den relevanten Zugriffspunkten. Diese Aufteilung reduziert die Komplexität im Kernel, ohne die Durchsetzung vollständig aus dem sicherheitsrelevanten Ausführungspfad herauszulösen.

2.6 Linux Security Modules und LSM-Hooks

Linux Security Modules (LSM) stellen ein Framework bereit, über das Sicherheitsmodule an sicherheitsrelevanten Stellen des Kernels Entscheidungen treffen können. Die Kernel-Dokumentation beschreibt LSM als Schnittstelle für Sicherheitsmodule wie SELinux, Smack, AppArmor oder TOMOYO [16]. Aus technischer Sicht werden dafür Hooks verwendet, die an bestimmten Punkten im Kernel-Ausführungspfad aufgerufen werden.

LSM-Hooks sind für Zugriffskontrolle besonders relevant, weil sie Operationen an Stellen erfassen können, an denen der Kernel ohnehin über die Zulässigkeit einer Operation entscheidet. Wright et al. beschreiben das LSM-Framework als allgemeine Sicherheitsunterstützung im Linux-Kernel, bei der Hooks Zugriff auf Kernelobjekte vermitteln und Rückgabewerte über Erfolg oder Fehler der Operation entscheiden können [17]. Damit eignen sich LSM-Hooks grundsätzlich als technische Grundlage für einen Reference-Monitor-ähnlichen Durchsetzungspunkt.

Für den Prototyp ist insbesondere der Hook `file_open` relevant. Er erlaubt eine Kontrolle beim Öffnen einer Datei und passt damit zur funktionalen Einschränkung dieser Arbeit auf Dateiöffnungen. LSM-Hooks bewerten jedoch jeweils konkrete Kerneloperationen. Ein beim Öffnen erlaubter Zugriff wird nicht automatisch erneut durch denselben Hook geprüft, wenn sich später Attribute ändern. Diese Eigenschaft erklärt, warum zusätzlich eine Überwachung bestehender Zugriffssituationen betrachtet werden muss.

2.7 eBPF und eBPF-LSM

eBPF ist ein Mechanismus, mit dem Programme in einer eingeschränkten Ausführungsumgebung im Linux-Kernel ausgeführt werden können. Die Programme werden vor der Ausführung durch den Kernel geprüft und können an verschiedene Ereignisse oder Hook-Typen gebunden werden. Historisch steht BPF für Berkeley Packet Filter. Die heute im Linux-Kernel verwendete erweiterte Form wird meist als eBPF bezeichnet; die Kernel-Dokumentation verwendet für das moderne Subsystem jedoch häufig weiterhin die Bezeichnung BPF. In dieser Arbeit wird deshalb eBPF verwendet, wenn der moderne Mechanismus für Programme, Maps und Verifier gemeint ist. Der Begriff BPF wird nur dort aufgegriffen, wo er in offiziellen Bezeichnungen wie `bpf()`-Systemaufruf, BPF-Dateisystem oder in der Kernel-Dokumentation selbst verwendet wird. Die Kernel-Dokumentation beschreibt BPF als im Kernel verwendetes Instruktionsformat und betont, dass BPF-

Programme auf definierte Hilfsfunktionen und Kernelobjekte wie Maps beschränkt sind [12].

eBPF-LSM verbindet eBPF mit dem LSM-Framework. Die Kernel-Dokumentation beschreibt LSM-BPF-Programme als Möglichkeit, LSM-Hooks zur Laufzeit durch privilegierte Benutzer zu instrumentieren, um systemweite Mandatory-Access-Control- oder Audit-Policies mit eBPF umzusetzen [13]. Damit kann ein eBPF-Programm an einem LSM-Hook ausgeführt werden und dort eine Entscheidung über die angeforderte Operation zurückgeben.

In den Grundlagen wird eBPF noch nicht als endgültige Lösung begründet, sondern als technischer Mechanismus eingeordnet. Für diese Arbeit ist eBPF interessant, weil es eine Ausführung im Kernel ermöglicht, ohne ein klassisches Kernelmodul zu entwickeln. Gleichzeitig ist eBPF durch Verifier, eingeschränkte Sprachelemente, begrenzten Stack, fehlende allgemeine Heap-Allokation und definierte Hilfsfunktionen deutlich stärker beschränkt als allgemeiner Kernelcode. Diese Einschränkungen müssen in der späteren Konzeption gegen die Vorteile abgewogen werden.

2.8 eBPF-Maps und gepinnte Maps

eBPF-Maps sind Datenstrukturen im Kernel, die von eBPF-Programmen und Userspace-Anwendungen gemeinsam genutzt werden können. Die Kernel-Dokumentation beschreibt Maps als generischen Speicher für unterschiedliche Datentypen, der unter anderem den Datenaustausch zwischen Kernelspace und Userspace ermöglicht [14]. Über den Systemaufruf `bpf()` können Maps erzeugt sowie Einträge gelesen, aktualisiert oder gelöscht werden [5].

Für Zugriffskontrolle sind Maps relevant, weil eBPF-Programme nicht beliebige komplexe Userspace-Datenstrukturen direkt verwenden können. Policies und Attribute müssen daher in eine feste, kernelgeeignete Struktur gebracht werden. Hash-Maps oder Arrays können beispielsweise verwendet werden, um Policy-Einträge, Attributwerte, Generationen oder Statusinformationen bereitzustellen. Der Kernel kann diese Werte während einer Zugriffskontrolle auslesen, während der Userspace sie bei Policy- oder Attributänderungen aktualisiert.

Gepinnte Maps erweitern diese Idee um einen stabilen Zugriffspfad im BPF-Dateisystem. Pinning erlaubt es, eBPF-Objekte wie Maps über einen Pfad zugänglich zu machen, sodass andere Prozesse sie später wieder öffnen können [2]. Dies ist für den Prototyp wichtig, weil nicht nur der Loader, sondern auch ein Monitor oder Administrationswerkzeug den aktuellen Zustand lesen können muss. Gepinnte Maps bilden damit eine zentrale Schnittstelle zwischen den getrennten Userspace-Komponenten und dem Kernelzustand.

2.9 eBPF-Verifier, Stack und Programmkomplexität

Bevor ein eBPF-Programm im Kernel ausgeführt werden darf, prüft der eBPF-Verifier dessen Sicherheit. Die Kernel-Dokumentation beschreibt die Prüfung als mehrstufigen Prozess, bei dem unter anderem Kontrollfluss, Registerzustände, Stackzugriffe und Speicherzugriffe analysiert werden [15]. Ein Programm wird nur geladen, wenn der Verifier zeigen kann, dass es die Sicherheitsregeln erfüllt.

Diese Prüfung hat direkte Auswirkungen auf die Gestaltung von eBPF-Programmen. Zugriffe auf Map-Werte, Stackbereiche oder Kontextdaten müssen für den Verifier nachvollziehbar sein. Der Verifier verfolgt unter anderem, ob Register initialisiert sind, welche Zeigerarten verwendet werden und ob Speicherzugriffe innerhalb bekannter Grenzen liegen. Unklare oder zu komplexe Zugriffsmuster können dazu führen, dass ein Programm abgelehnt wird, auch wenn die fachliche Logik korrekt erscheint. Darüber hinaus muss der Kontrollfluss eines eBPF-Programms für den Verifier analysierbar bleiben. Schleifen dürfen nicht unbeschränkt sein, Rekursion ist nicht als allgemeines Programmiermittel verfügbar, und Speicherzugriffe müssen durch bekannte Grenzen abgesichert sein. Auch der Zugriff auf Kernelinformationen erfolgt nicht beliebig, sondern über den Hook-Kontext, Maps oder für den jeweiligen Programmtyp zugelassene Hilfsfunktionen.

Zusätzlich bestehen praktische Begrenzungen bei Stack und Programmkomplexität. Die BPF-Dokumentation nennt für Programme ein Stacklimit von 512 Byte und beschreibt interne Grenzen des Verifiers, darunter die Anzahl der während der Analyse betrachteten Instruktionen und Zustände [12]. Die offizielle Aya-Dokumentation beschreibt eBPF als eingeschränkte Laufzeitumgebung und weist darauf hin, dass eBPF-Programme nur 512 Byte Stack besitzen; bei Programmen, die Tail Calls verwenden, kann der verfügbare Stack auf 256 Byte begrenzt sein [11]. Der Stack eignet sich damit nur für kleine lokale Hilfsdaten, etwa Schlüsselstrukturen für Map-Zugriffe oder wenige temporäre Werte. Größere lokale Puffer, dynamisch wachsende Listen oder Zeichenketten können dort nicht sinnvoll abgelegt werden. Ein allgemeiner Heap, wie er in Userspace-Programmen für dynamische Datenstrukturen wie Listen, Strings oder Objekte genutzt wird, steht eBPF-Programmen nicht zur Verfügung. Veränderliche oder größere Zustände müssen daher typischerweise in eBPF-Maps ausgelagert werden. Für Rust-basierte eBPF-Programme ergeben sich daraus weitere Einschränkungen. Die Standardbibliothek kann im eBPF-Programm nicht verwendet werden; stattdessen steht nur `core` zur Verfügung. Funktionen und Traits, die auf `core::fmt` beruhen, etwa `Display` oder `Debug`, sind nicht nutzbar. Da kein Heap verfügbar ist, können auch `alloc` und darauf aufbauende Collections nicht eingesetzt werden. Außerdem darf ein eBPF-Programm nicht auf Panics angewiesen sein, weil die eBPF-Laufzeit weder Stack-Unwinding noch eine Abort-Instruktion unterstützt. Ein eBPF-Programm besitzt zudem keine gewöhnliche `main`-Funktion, sondern wird über einen programmspezifischen Einstiegspunkt an einen Hook gebunden. Da eBPF-Programme häufig direkt Kernel-Speicher oder Hook-Kontextdaten lesen, ist außerdem ein relevanter Teil des Codes als `unsafe` zu behandeln [11]. Für die Arbeit bedeutet dies, dass komplexe Policy-Logik möglichst im Userspace vorbereitet werden sollte, während das eBPF-Programm nur eine kompakte, vorher strukturierte Repräsentation auswertet.

2.10 Aya als Rust-Framework für eBPF

Aya ist ein Rust-Framework zur Entwicklung von eBPF-Anwendungen. Die offizielle Aya-Dokumentation beschreibt Aya als Möglichkeit, eBPF-Programme mit Rust zu entwickeln und dabei Cargo, Rust-Strukturen sowie gemeinsame Codeanteile zwischen Userspace und eBPF-Programm zu nutzen [11]. Neben Rust-basierten Frameworks existieren etablierte eBPF-Werkzeuge im C- und Skriptsprachenumfeld. `libbpf` ist eine C-Bibliothek für die Entwicklung und das Laden von BPF- beziehungsweise eBPF-Programmen, während `BCC` (BPF Compiler Collection) ein Toolkit für BPF-basierte Analyse-, Tracing- und

Monitoring-Werkzeuge ist [? ?]. Aya stellt dazu eine Rust-basierte Alternative dar.

Für den Prototyp ist Aya vor allem deshalb relevant, weil Userspace-Komponenten und eBPF-Programm in derselben Programmiersprache entwickelt werden können. Gemeinsame Datentypen können in einer gemeinsamen Bibliothek definiert werden, wodurch sich die Gefahr verringert, dass Userspace und Kernel-space unterschiedliche Annahmen über die Struktur von Map-Schlüsseln oder Map-Werten treffen. Gerade bei Policies und Attributen ist diese Konsistenz wichtig, weil dieselben binären Strukturen auf beiden Seiten der Map-Schnittstelle interpretiert werden.

Gleichzeitig hebt Aya die Einschränkungen von eBPF nicht auf. Auch mit Rust gelten die Vorgaben des eBPF-Verifiers, die Stackbegrenzung, der Verzicht auf Heap-Allokation im eBPF-Programm und die Beschränkung auf unterstützte Hilfsfunktionen. Aya ist daher als Entwicklungswerkzeug zu verstehen, nicht als fachliches Zugriffskontrollmodell. Die Entscheidung für Aya betrifft die spätere Implementierung, während die fachlichen Anforderungen weiterhin aus ASBAC und der Kernel-Durchsetzung abgeleitet werden.

2.11 Dynamische Attribute und Policy-Generationen

Dynamische Attribute sind Attributwerte, die sich zur Laufzeit ändern können. Im Unterschied zu statischen Eigenschaften, etwa einem festen Dateipfad, können dynamische Attribute zeitabhängig oder durch externe Ereignisse beeinflusst sein. Beispiele sind Uhrzeit, Systemzustand oder vom Nutzer frei definierte Subjektattribute. Die Idee veränderlicher Attribute knüpft an Usage-Control-Modelle an, in denen Attributmutabilität als ausdrückliches Merkmal betrachtet wird [3].

Für den Prototyp bedeutet dies, dass Attribute nicht nur beim Start einmalig geladen werden dürfen. Änderungen müssen vom Userspace erkannt, validiert und in die kernel-geeignete Repräsentation übertragen werden. Da eBPF-Programme zur Laufzeit auf Maps zugreifen können, bieten Maps eine geeignete Schnittstelle, um aktuelle Attributwerte für Zugriffsentscheidungen bereitzustellen [14]. Die Attributnamen und Werte müssen dabei im Userspace so verarbeitet werden, dass das eBPF-Programm mit festen Schlüssel- und Wertstrukturen arbeiten kann.

Policy-Generationen beschreiben zusammengehörige gültige Stände von Policies. Dieser Begriff ist für den Prototyp wichtig, weil Policy-Dateien im Userspace geparkt und validiert werden, bevor sie für Zugriffsentscheidungen verwendet werden. Eine neue Generation sollte erst dann aktiv werden, wenn alle dazugehörigen Map-Einträge vollständig erzeugt und konsistent bereitgestellt wurden. Dadurch wird vermieden, dass der Kernel während einer Aktualisierung auf einen unvollständigen oder widersprüchlichen Policy-Zustand zugreift.

2.12 Überwachung bestehender Zugriffe und File Descriptors

Ein Dateizugriff in Linux wird häufig über einen File Descriptor repräsentiert. Der Systemaufruf `open()` liefert bei Erfolg einen nichtnegativen File Descriptor zurück, der in nachfolgenden Systemaufrufen wie `read()`, `write()` oder `lseek()` als Referenz auf die geöffnete Datei verwendet wird [6]. Damit entsteht nach dem Öffnen eine bestehende Zugriffssituation, die unabhängig vom ursprünglichen Pfad weiter über den File Descriptor genutzt werden kann.

Für eine Zugriffskontrolle über `file_open` ist das wichtig, weil der LSM-Hook beim Öffnen der Datei greift. Wenn sich danach entscheidungsrelevante Attribute ändern, wird derselbe bereits geöffnete File Descriptor nicht automatisch erneut durch `file_open` bewertet. Eine ASBAC-orientierte Zugriffskontrolle muss deshalb zusätzlich betrachten, wie bestehende File Descriptors überwacht und bei geänderter Entscheidungsgrundlage behandelt werden können.

Linux stellt über `/proc/<pid>/fd` Informationen über die geöffneten File Descriptors eines Prozesses bereit. Die man-pages beschreiben die Einträge in diesem Verzeichnis als symbolische Links auf die geöffneten Dateien oder andere referenzierte Objekte; der Zugriff darauf unterliegt eigenen Berechtigungsprüfungen [7]. Für den Prototyp kann ein Userspace-Monitor diese Informationen nutzen, um zu erkennen, welcher Prozess über welchen File Descriptor auf welche Datei zugreift. Wird eine bestehende Zugriffssituation durch neue Attribute oder Policies unzulässig, kann der Monitor gezielt den betroffenen Zugriff behandeln, ohne notwendigerweise alle Zugriffe desselben Prozesses zu beenden.

3 Anforderungsanalyse

In diesem Kapitel werden die Anforderungen an den zu entwickelnden Prototypen beschrieben. Die Anforderungen ergeben sich aus der Zielsetzung, zu untersuchen, wie Attribute Stream-Based Access Control im Linux-Kernel prototypisch umgesetzt werden kann. Sie beschreiben das Verhalten, das der Prototyp im Rahmen der Arbeit zeigen soll.

3.1 Funktionale Anforderungen

Funktionale Anforderungen beschreiben, welche fachlichen Funktionen der Prototyp bereitstellen muss. Sie sind im Regelfall direkt testbar, da geprüft werden kann, ob eine Funktion vorhanden ist und das erwartete Verhalten zeigt. Im Kontext dieser Arbeit betreffen sie insbesondere das Einlesen und Verarbeiten von Policies, die Zugriffskontrolle über LSM-Hooks sowie die nachträgliche Überwachung bestehender Zugriffe.

FA-01 Policy-Einlesen: Der Prototyp muss ASBAC-Policies aus einer definierten Quelle im Userspace einlesen können.

Akzeptanzkriterium: Eine im Policy-Verzeichnis abgelegte Policy wird erkannt und für die weitere Verarbeitung bereitgestellt.

FA-02 Policy-Verwaltung: Der Prototyp muss Änderungen an Policies erkennen und aktualisierte Policies in die Zugriffskontrolle übernehmen können.

Akzeptanzkriterium: Wird eine Policy geändert, hinzugefügt oder entfernt, wird diese Änderung vom System verarbeitet.

FA-04 Zugriffskontrolle bei Dateiöffnungen: Der Prototyp muss Dateiöffnungen über den Linux-Security-Module-Hook `file_open` abfangen können.

Akzeptanzkriterium: Das Öffnen einer durch eine Policy erfassten Datei wird durch den LSM-Hook `file_open` kontrolliert.

FA-05 Entscheidungsfindung anhand von Policies: Der Prototyp muss Zugriffsentscheidungen auf Grundlage der geladenen Policies treffen können.

Akzeptanzkriterium: Ein Zugriff wird abhängig von einer Policy erlaubt oder verweigert.

FA-06 Unterstützung dynamischer Attribute: Der Prototyp muss dynamische Attribute, beispielsweise Zeitinformationen oder externe Zustandsinformationen, in Zugriffsentscheidungen einbeziehen können.

Akzeptanzkriterium: Eine Änderung eines dynamischen Attributs kann zu einer veränderten Zugriffsentscheidung führen.

FA-07 Überwachung bestehender Dateizugriffe: Der Prototyp muss bereits bestehende Dateizugriffe überwachen können, wenn sich entscheidungsrelevante Policies oder Attribute zur Laufzeit ändern. Als bestehender Zugriff gilt im Rahmen des Prototyps ein geöffneter File Descriptor eines laufenden Prozesses auf eine durch eine Policy erfasste Datei. Der Monitor muss erkennen können, welcher Prozess über welchen

File Descriptor auf welche Datei zugreift und ob dieser Zugriff nach der aktuellen Policy- und Attributlage weiterhin zulässig ist.

Akzeptanzkriterium: Öffnet ein Prozess eine Datei zunächst zulässig und wird der Zugriff anschließend durch eine Änderung eines entscheidungsrelevanten Attributs unzulässig, erkennt der Monitor diese Verletzung und kann Prozess-ID, File Descriptor und betroffenen Dateipfad ausgeben.

FA-08 Enforcement bei unzulässig gewordenen Dateizugriffen: Der Prototyp muss einen bestehenden Dateizugriff beenden können, wenn dieser aufgrund einer Policy- oder Attributänderung nicht mehr zulässig ist. Im Rahmen des Prototyps erfolgt das Enforcement für Dateizugriffe durch das Schließen des betroffenen File Descriptors des jeweiligen Prozesses. Dabei darf nur der File Descriptor geschlossen werden, der den unzulässig gewordenen Zugriff repräsentiert.

Akzeptanzkriterium: Besitzt ein Prozess mehrere geöffnete File Descriptors und wird nur einer dieser Zugriffe durch eine Attribut- oder Policyänderung unzulässig, schließt der Prototyp nur diesen betroffenen File Descriptor. Andere zulässige File Descriptors desselben Prozesses bleiben geöffnet.

FA-09 Administrationsschnittstelle: Der Prototyp soll eine einfache Administrationschnittstelle bereitstellen, über die zentrale Informationen zum Zustand des Prototyps lesend eingesehen werden können.

Akzeptanzkriterium: Ein Administrator kann zentrale Informationen zum Zustand des Prototyps abrufen, ohne über die Administrationsschnittstelle Policies, Attribute oder Kernelzustand zu verändern.

FA-10 Gültige Policy-Generationen: Der Prototyp muss sicherstellen, dass nur erfolgreich erzeugte und validierte Generationen von Policies für Zugriffsentscheidungen verwendet werden. Fehlerhafte, unvollständige oder nicht erfolgreich verarbeitete Policy-Generationen dürfen nicht als aktive Entscheidungsgrundlage gelten.

Akzeptanzkriterium: Eine neue Policy-Generation wird erst dann aktiviert, wenn sie vollständig verarbeitet und erfolgreich validiert wurde. Andernfalls bleibt die zuletzt gültige Policy-Generation aktiv.

FA-11 Eingabvalidierung: Der Prototyp muss Eingaben aus dem Userspace validieren, bevor diese im Kernel für Zugriffsentscheidungen verwendet werden.

Akzeptanzkriterium: Ungültige oder manipulierte Eingaben werden erkannt und nicht als Entscheidungsgrundlage im Kernel verwendet.

FA-12 beliebige Attributnamen: In Policies muss es für den Nutzer möglich sein, Attributnamen frei zu vergeben, ohne dass diese vorher im Prototyp fest definiert sein müssen. Zulässige Attributnamen dürfen nicht leer sein und bestehen aus ASCII-Buchstaben (A-Z, a-z), Ziffern (0-9), Unterstrich (_) und Bindestrich (-).

Akzeptanzkriterium: Eine Policy mit einem gültigen, zuvor nicht bekannten Attributnamen, beispielsweise `subject.position == engineer`, wird erfolgreich verarbeitet. Eine Policy mit einem Attributnamen außerhalb des zulässigen Zeichensatzes wird abgelehnt.

3.2 Operative Anforderungen

Operative Anforderungen beschreiben, wie gut der Prototyp seine Funktionen im Betrieb erfüllen soll. Sie beziehen sich auf Eigenschaften, die für Nutzung, Betrieb und Evaluation sichtbar sind, beispielsweise Stabilität, Beobachtbarkeit, Nachvollziehbarkeit, Performance und Reproduzierbarkeit.

OA-01 Stabilität: Der Prototyp darf die Stabilität des Linux-Kernels nicht unnötig gefährden.

Akzeptanzkriterium: Fehlerhafte oder ungültige Eingaben aus dem Userspace führen nicht zu einem Kernel-Absturz.

OA-02 Beobachtbarkeit und Nachvollziehbarkeit: Der Prototyp soll zentrale Zustände und Entscheidungen so sichtbar machen, dass Betrieb und Evaluation nachvollziehen können, welche Policies, Attribute oder Zustände eine Zugriffsentscheidung beeinflusst haben.

Akzeptanzkriterium: Für zentrale Zugriffsentscheidungen ist erkennbar, welche Policy oder welcher Zustand die Entscheidung beeinflusst hat.

OA-03 Performance: Die zusätzlichen Prüfungen durch den Prototyp sollen den kontrollierten Zugriff nicht unverhältnismäßig verlangsamen.

Akzeptanzkriterium: Die Auswirkungen auf die Ausführungszeit werden in der Evaluation betrachtet.

OA-04 Reproduzierbarkeit: Die Evaluation des Prototyps soll unter dokumentierten Bedingungen wiederholbar sein.

Akzeptanzkriterium: Testumgebung, Policies und Testszenarien werden so beschrieben, dass die Ergebnisse nachvollzogen werden können.

3.3 Entwicklungsbezogene Anforderungen

Entwicklungsbezogene Anforderungen beschreiben Eigenschaften, die vor allem für die Entwicklung und spätere Erweiterung des Prototyps relevant sind. Sie sind für Nutzer nicht immer unmittelbar sichtbar, beeinflussen aber, wie verständlich, änderbar und erweiterbar das System bleibt.

EA-01 Wartbarkeit: Die Implementierung soll modular aufgebaut sein, sodass Userspace-Komponenten, Kernel-Komponenten und Kommunikationsschnittstelle getrennt betrachtet werden können.

EA-02 Erweiterbarkeit: Der Prototyp soll so entworfen werden, dass weitere Attribute, Policies oder LSM-Hooks ergänzt werden können.

EA-03 Begrenzter Kernel-Anteil: Komplexe Policy-Logik soll möglichst im Userspace verbleiben, um die Komplexität im Kernel zu reduzieren.

4 Konzeption

4.1 praktische Einschränkungen von eBPF

Aus den beschriebenen eBPF-Einschränkungen folgt, dass der Kernelanteil des Prototyps keine frei strukturierte Policy-Sprache auswerten kann. Textuelles Parsen, dynamische Datenstrukturen und komplexe Auswertungslogik würden sowohl den begrenzten Stack als auch die Anforderungen des Verifiers unnötig belasten. Die Policy-Dateien werden deshalb im Userspace gelesen, validiert und in feste, kernelgeeignete Map-Einträge übersetzt.

Für die Policy-Sprache bedeutet dies, dass Bedingungen nur in einer begrenzten, vorher festgelegten Struktur unterstützt werden. Der Kernel verarbeitet keine Zeichenketten dynamisch, sondern vergleicht feste Felder, numerische Werte oder vorberechnete Hashwerte. Attribute werden ebenfalls nicht als frei wachsende Listen im eBPF-Programm gehalten, sondern über geordnete Schlüssel-Wert-Einträge in eBPF-Maps bereitgestellt. Dadurch kann das eBPF-Programm zur Laufzeit kompakte Lookups ausführen, ohne selbst Speicher dynamisch anzufordern. Auch die Rust-spezifischen Einschränkungen wirken sich auf den Entwurf aus. Der Kernelanteil darf keine Standardbibliothek, keine heapbasierten Collections und keine formatierenden Ausgaben über `Display` oder `Debug` voraussetzen. Fehlerfälle müssen daher über explizite Rückgabewerte oder einfache Kontrollpfade behandelt werden, anstatt sich auf Panics zu stützen. Debug-Ausgaben im eBPF-Code müssen auf dafür geeignete, einfache Hilfsmechanismen beschränkt bleiben.

Die Programmlogik muss außerdem so aufgeteilt werden, dass einzelne eBPF-Programme klein genug für Verifier und Stack bleiben. Dies betrifft insbesondere die Auswertung mehrerer Policy-Gruppen. Komplexere Prüfungen werden daher über vorbereitete Map-Strukturen und, soweit erforderlich, über Tail Calls verteilt. Diese Aufteilung reduziert die Programmkomplexität pro eBPF-Programm, führt aber gleichzeitig dazu, dass die Schnittstellen zwischen den Programmteilen und die Stack-Nutzung besonders sorgfältig begrenzt werden müssen.

4.2 Architektur

Die Architektur des Prototyps ist in zwei zentrale Bereiche unterteilt: den Userspace und den Kernel-space. Diese Trennung orientiert sich an der grundsätzlichen Struktur von Linux-Systemen und ermöglicht es, komplexe Logik zur Policy-Verarbeitung außerhalb des Kernels auszuführen, während sicherheitskritische Zugriffsentscheidungen direkt im Kernel umgesetzt werden.

- **Userspace:** Im Userspace befinden sich die Komponenten zur Verwaltung und Verarbeitung von Policies. Dieser unterteilt sich in das Hauptprogramm, einen Attributloader, einen Monitor und der Policyverwaltung. Das Hauptprogramm lädt das Kernelprogramm und startet dieses. Der Attributloader lädt Streamingattribute in den Kernel-space. Die Policyverwaltung liest die textuellen Policy-Dateien, parst diese und übersetzt sie in kernelgeeignete Map-Einträge. Der Monitor überwacht Dateizugriffe im Userspace und prüft mögliche Policy-Verletzungen. Durch eine nachträgliche Durchsetzungsmaßnahme kann der Prozess bei einer Policy-Verletzung beendet oder der Dateizugriff durch das Schließen des File Descriptors beendet werden.

- **Kernelspace:** Im Kernelspace wird ein Linux Security Module integriert. Dieses greift über LSM-Hooks in sicherheitsrelevante Operationen ein und trifft Zugriffsentscheidungen auf Basis der vom Userspace bereitgestellten Policies. Hierbei wertet das LSM auch Streamingattribute, welche vom Userspace aus in den Kernel gestreamt werden aus.

Die Kommunikation zwischen Userspace und Kernelspace bildet dabei eine zentrale Schnittstelle des Systems. Sie dient dazu, Policies, Entscheidungen oder Attribute zwischen beiden Bereichen auszutauschen. Dadurch kann die Policy-Logik im Userspace verbleiben, während der Kernel die unmittelbare Durchsetzung der Zugriffskontrolle bei Dateizugriffen übernimmt. Für Zugriffe, die bereits durch einen LSM-Hook erlaubt wurden, erfolgt keine automatische erneute Bewertung durch denselben Hook. Diese nachträgliche Überwachung wird durch den Monitor übernommen.

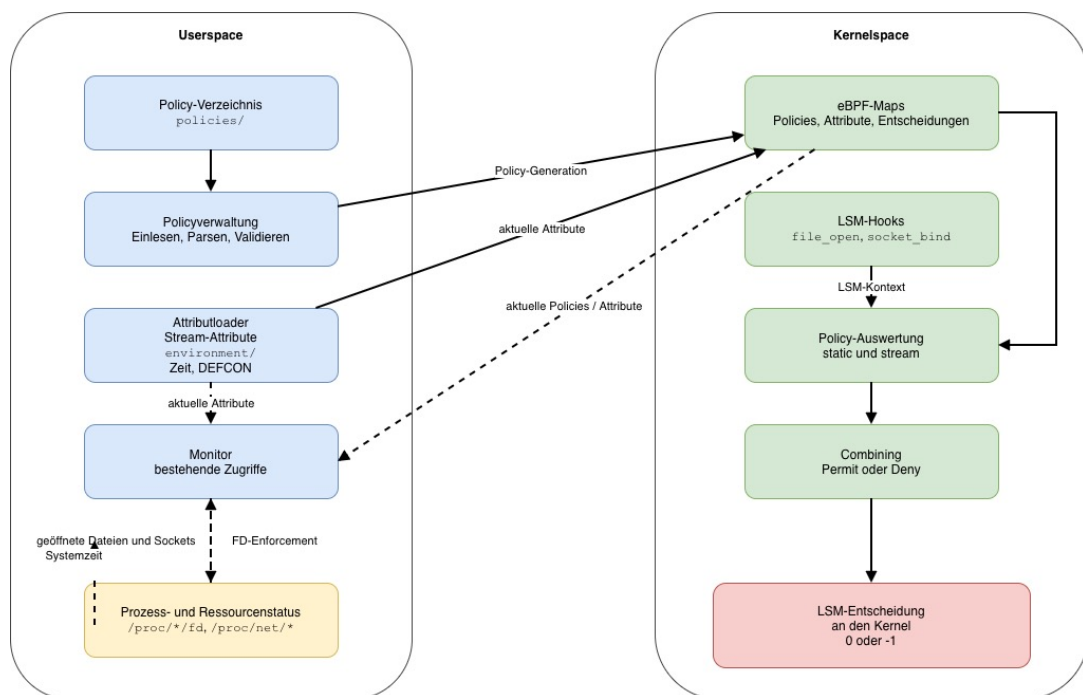


Abbildung 1: Konzeptionelle Architektur des Prototyps

4.3 Datenfluss

Der Datenfluss des Prototyps lässt sich in vier Teilbereiche unterteilen. Diese Teilbereiche beschreiben, wie Policies in das System gelangen, wie dynamische Attribute aktualisiert werden, wie ein Zugriff im Kernel bewertet wird und wie bereits bestehende Zugriffe nachträglich überwacht werden.

4.3.1 Policy-Datenfluss

Policies werden im Userspace in einem dafür vorgesehenen Policy-Verzeichnis abgelegt. Die Policyverwaltung überwacht dieses Verzeichnis und erkennt, wenn Policies hinzugefügt, geändert oder entfernt werden. Nach einer Änderung werden die vorhandenen

Policy-Dateien eingelesen und in kernelgeeignete Map-Einträge übersetzt. Dabei wird geprüft, ob die Policies syntaktisch und semantisch für den Prototyp verarbeitbar sind.

Erst wenn diese Verarbeitung erfolgreich abgeschlossen ist, werden die Policies als neue Policy-Generation in den Kernel übertragen. Die Übertragung erfolgt über gepinnte eBPF-Maps, die als gemeinsame Datenstruktur zwischen Userspace und Kernelspace dienen. Durch das Generationenmodell wird verhindert, dass der Kernel während einer Zugriffsentscheidung eine nur teilweise aktualisierte Policy-Menge verwendet. Schlägt die Verarbeitung oder Übertragung fehl, bleibt die zuletzt gültige Policy-Generation aktiv.

4.3.2 Datenfluss dynamischer Attribute

Neben den Policies werden dynamische Attribute aus dem Userspace in den Kernelspace übertragen. Dynamische Attribute sind Werte, die sich zur Laufzeit ändern und deshalb regelmäßig aktualisiert werden müssen. Im Prototyp betrifft dies insbesondere Zeitinformationen sowie vom Nutzer frei wählbare Attribute (vgl. FA-12).

Die Userspace-Komponente ermittelt diese Werte beziehungsweise liest sie aus einer definierten Quelle ein. Anschließend werden die aktuellen Werte in dafür vorgesehene eBPF-Maps geschrieben. Der Kernel kann diese Werte bei der Auswertung von Policies lesen und in die Zugriffsentscheidung einbeziehen.

4.3.3 Zugriffsentscheidungen im Kernel

Wenn ein kontrollierter Zugriff stattfindet, wird im Kernel ein LSM-Hook ausgelöst. Bei einem Dateizugriff ist dies beispielsweise der Hook `file_open`. Das eBPF-Programm liest aus dem Hook-Kontext die für die Entscheidung relevanten Informationen aus. Dazu gehören unter anderem das Subjekt, die angefragte Ressource und der aktuelle Prozesskontext.

Auf Grundlage dieser Zugriffsdaten werden die zugehörigen Policies im Kernel ausgewertet. Dabei werden nur Policies berücksichtigt, die für den jeweiligen Hook vorgesehen und auf die konkrete Zugriffssituation anwendbar sind. Eine Policy ist beispielsweise nur dann anwendbar, wenn Benutzer, Prozessname und Ressource zu der aktuellen Anfrage passen. Erst wenn eine Policy anwendbar ist, kann sie eine Aussage in Form von *Permit* oder *Deny* treffen.

Die Zwischenergebnisse der Policy-Auswertung werden anschließend zusammengeführt. Das Combining entscheidet, ob der Zugriff insgesamt erlaubt oder verweigert wird. Führt das Ergebnis zu einer Verweigerung, gibt das eBPF-LSM-Programm einen Fehlerwert an den Kernel zurück. Der Kernel unterbindet daraufhin den Zugriff.

4.3.4 Monitoring bestehender Zugriffe

LSM-Hooks prüfen einen Zugriff zu dem Zeitpunkt, an dem das jeweilige Kernelereignis auftritt. Bei streambasierten Policies kann sich die Entscheidungslage jedoch nachträglich ändern, beispielsweise wenn sich die Uhrzeit oder andere Attribute ändern. Ein bereits geöffneter Zugriff wird dadurch nicht automatisch erneut durch denselben LSM-Hook geprüft.

Aus diesem Grund enthält der Prototyp zusätzlich einen Monitor im Userspace. Der Monitor betrachtet regelmäßig aktive Prozesse und deren aktuell verwendete Ressourcen.

Dabei prüft er, ob eine bestehende Zugriffssituation mit den derzeit aktiven Policies vereinbar ist. Wird eine Verletzung erkannt, kann der Monitor ein nachträgliches Enforcement auslösen. Im Fall von Dateizugriffen kann dies beispielsweise bedeuten, dass betroffene File Descriptors eines Prozesses geschlossen werden.

Der Monitor ergänzt damit die kernelbasierte Zugriffskontrolle. Während die LSM-Hooks neue Zugriffe unmittelbar prüfen, adressiert der Monitor das Problem bereits bestehender Zugriffe bei nachträglich geänderten Bedingungen.

4.4 Komponenten

Die Architektur des Prototyps setzt sich aus mehreren Komponenten zusammen, die jeweils eine abgegrenzte Aufgabe innerhalb der Zugriffskontrolle übernehmen. Im Folgenden werden die wichtigsten Komponenten beschrieben.

4.4.1 Userspace-Komponenten

Im Userspace befinden sich die Komponenten, die Policies verwalten, dynamische Attribute bereitstellen und bestehende Zugriffssituationen überwachen. Dadurch verbleiben komplexere Verarbeitungsschritte außerhalb des Kernels.

Policies Policies beschreiben die Zugriffsvorgaben des Systems. Sie legen fest, welches Subjekt unter welchen Bedingungen auf welche Ressource zugreifen darf oder nicht zugreifen darf. Im Prototyp liegen sie als Textdateien im Policy-Verzeichnis `policies/`. Policies sind damit keine ausführbare Programmlogik, sondern Eingabedaten für die Policyverwaltung und die spätere Auswertung im Kernel-space.

Policyverwaltung Die Policyverwaltung bildet die Schnittstelle zwischen den menschenlesbaren Policy-Dateien und den im Kernel verwendbaren Map-Einträgen. Sie überwacht das Verzeichnis `policies/`, liest die dort abgelegten Policies ein, verarbeitet sie und validiert sie vor der Aktivierung. Nach erfolgreicher Validierung übersetzt der Userspace die Policy-Dateien in kernelgeeignete Map-Einträge.

Da Policies während des Betriebs geändert werden können, muss verhindert werden, dass der Kernel eine nur teilweise aktualisierte Policy-Menge verwendet. Dazu aktiviert die Policyverwaltung Policies als atomare Einheit. Dies wird durch ein Generationensystem gewährleistet: Eine neue Policy-Generation wird zunächst vollständig im Userspace vorbereitet und validiert. Erst danach wird sie in den Kernel-space übertragen und als gültige Generation aktiviert. Schlägt dieser Vorgang fehl, bleibt die zuletzt gültige Generation aktiv.

Die Policyverwaltung umfasst damit gelesene Policy-Dokumente, geparste Policies, kernelgeeignete Map-Einträge, Policy-Generationen sowie aktive und zuletzt gültige Policy-Versionen.

Attributloader Der Attributloader stellt dynamische Kontextinformationen für die Zugriffskontrolle bereit. Diese Attribute sind nicht notwendigerweise Bestandteil eines einzelnen Zugriffsvorgangs, können aber die Entscheidung beeinflussen. Im Prototyp betrifft dies insbesondere die aktuelle Zeit sowie externe Attribute aus dem Verzeichnis

environment/. Der Attributloader schreibt diese Werte in den Kernelspace, damit sie dort bei der Auswertung von Streampolicies berücksichtigt werden können.

Der Attributloader trifft selbst keine Zugriffsentscheidung. Seine Aufgabe besteht darin, die Entscheidungsgrundlage aktuell zu halten.

Monitor Der Monitor ergänzt die kernelbasierte Zugriffskontrolle um die nachträgliche Überwachung bereits bestehender Zugriffssituationen. Dies ist notwendig, weil ein LSM-Hook einen Zugriff nur zum Zeitpunkt des jeweiligen Kernelereignisses prüft. Ändern sich danach dynamische Attribute, wird ein bereits bestehender Zugriff nicht automatisch erneut durch denselben Hook geprüft.

Der Monitor betrachtet deshalb regelmäßig laufende Prozesse und deren verwendete Ressourcen. Dazu nutzt er den Prozess- und Ressourcenstatus des Betriebssystems, beispielsweise Informationen über offene File Descriptors. Diese Informationen werden gegen die aktuell aktiven Policies geprüft. Wird eine Verletzung erkannt, kann ein nachträgliches Enforcement ausgelöst werden. Bei Dateizugriffen kann dies beispielsweise bedeuten, dass ein betroffener File Descriptor geschlossen und damit der Zugriff auf eine bereits geöffnete Ressource beendet wird.

Der Monitor ersetzt damit nicht die Zugriffskontrolle im Kernelspace, sondern ergänzt sie für Zugriffssituationen, die nach der ursprünglichen Hook-Entscheidung unzulässig werden.

4.4.2 Kernelspace-Komponenten

LSM Die LSM-Komponente bildet den kernelbasierten Durchsetzungspunkt des Prototyps. Sie greift über LSM-Hooks in sicherheitsrelevante Operationen ein und erhält dadurch Zugriff auf die für die Entscheidung relevanten Zugriffsdaten. Auf dieser Grundlage werden die im Kernelspace vorliegenden Policies ausgewertet. Das Ergebnis dieser Auswertung entscheidet, ob der jeweilige Zugriff erlaubt oder verweigert wird.

4.4.3 Administrationskomponente

Administrationstool Das Administrationstool dient der Diagnose des Prototyps. Es ermöglicht, zentrale Informationen zum Zustand des Systems, über geladene Policies und Attribute lesend einzusehen. Zur Gewährleistung der Konsistenz des Systems wird hier auf eine schreibende Funktion verzichtet.

4.5 Datenobjekte

Neben den aktiven Komponenten besitzt der Prototyp mehrere zentrale Datenobjekte. Diese Datenobjekte beschreiben den Zustand des Systems und bilden die Grundlage für Policy-Verarbeitung, Zugriffsauswertung und Monitoring. Im Folgenden wird zwischen Datenobjekten im Userspace und Datenobjekten im Kernelspace unterschieden.

4.5.1 Datenobjekte im Userspace

Im Userspace liegen die Datenobjekte, die für die Verwaltung, Vorbereitung und nachträgliche Überwachung von Policies benötigt werden. Dazu gehören zunächst die Policy-

Dateien im Verzeichnis `policies/`. Sie stellen die menschenlesbare Beschreibung der Zugriffsvorgaben dar.

Während der Verarbeitung entstehen daraus weitere Datenobjekte: gelesene Policy-Dokumente, geparte Policies und kernelgeeignete Map-Einträge. Diese Zwischenformen sind notwendig, um die textuellen Policies in eine strukturierte Form zu überführen, die später in eBPF-Maps im Kernspace übertragen werden kann. Zusätzlich verwaltet der Userspace Policy-Generationen. Eine Policy-Generation beschreibt eine zusammengehörige Menge von Policies, die gemeinsam aktiviert oder verworfen wird.

Weitere Datenobjekte im Userspace sind dynamische Attribute wie Zeitinformationen oder externe Zustandswerte aus dem Verzeichnis `environment/`. Diese Werte werden vom Attributloader aktualisiert und in den Kernspace übertragen.

Für den Monitor sind außerdem Prozess- und Ressourceninformationen relevant. Dazu zählen laufende Prozesse, Benutzer-IDs, Prozessnamen, offene File Descriptors, geöffnete Dateien und aktive Sockets. Diese Informationen stammen aus Betriebssystemquellen wie `/proc/<PID>/fd` und werden verwendet, um bestehende Zugriffssituationen gegen die aktiven Policies zu prüfen.

4.5.2 Datenobjekte im Kernspace

Im Kernspace liegen die Datenobjekte, die für die unmittelbare Zugriffskontrolle an den LSM-Hooks benötigt werden. Dazu gehören die geladenen statischen Policies und Streampolicies. Sie werden nach erfolgreicher Validierung aus dem Userspace übertragen und in eBPF-Maps abgelegt.

Neben den Policies befinden sich auch dynamische Attribute im Kernspace. Dazu zählen insbesondere die aktuelle Zeit und externe Zustandswerte, die vom Attributloader bereitgestellt werden. Diese Attribute dienen der Auswertung von Streampolicies.

Bei einem Zugriff erzeugt der Kernspace außerdem Zugriffsdaten. Diese beschreiben die konkrete Anfrage, beispielsweise das Subjekt, den Prozessnamen, die betroffene Ressource oder Socket-Informationen. Auf Grundlage dieser Zugriffsdaten werden die geladenen Policies ausgewertet.

Schließlich existieren im Kernspace Zwischenergebnisse der Entscheidungsfindung. Sie halten fest, ob während der Auswertung eine anwendbare Permit- oder Deny-Policy gefunden wurde. Diese Zwischenergebnisse werden im Combining zusammengeführt und bestimmen die finale Entscheidung des LSM-Hooks.

4.6 Entwurfsentscheidungen

4.6.1 Trennung von Userspace und Kernspace

Das Reference-Monitor-Konzept bildet eine grundlegende theoretische Basis für die Umsetzung von Zugriffskontrollmechanismen. Nach Anderson muss ein Reference Monitor manipulationssicher und verifizierbar sein. Weiterhin muss eine vollständige Überprüfung aller Zugriffe auf geschützte Ressourcen gewährleistet sein [1]. Wright et al. beschreiben, dass das LSM-Framework Sicherheits-Hooks an sicherheitskritischen Stellen des Kernels einfügt und dass diese Hooks Rückgabewerte liefern, welche über Erfolg oder Fehler der angeforderten Operation entscheiden. Daraus folgt, dass die Zugriffskontrollentscheidung innerhalb des Kernel-Ausführungspfades erfolgen muss [17].

Daraus folgt jedoch nicht, dass die gesamte Policy-Verarbeitung im Kernelspace erfolgen muss. Insbesondere das Einlesen, Parsen und Validieren textueller Policy-Dateien ist vergleichsweise komplex und fehleranfällig. Diese Verarbeitungsschritte eignen sich daher besser für den Userspace, da Fehler dort nicht unmittelbar die Stabilität des Kernels gefährden. Der Kernelspace wird im Prototyp auf die sicherheitskritische Auswertung einer bereits vorbereiteten Policy-Repräsentation beschränkt.

Die gewählte Architektur trennt daher zwischen Policy-Verarbeitung im Userspace und Zugriffsdurchsetzung im Kernelspace. Der Userspace übernimmt die Verwaltung der Policies, die Bereitstellung dynamischer Attribute und die Überwachung bereits bestehender Zugriffssituationen. Der Kernelspace übernimmt die unmittelbare Entscheidung an den LSM-Hooks. Damit wird die Komplexität im Kernel reduziert, während die Zugriffskontrolle weiterhin im sicherheitskritischen Ausführungspfad des Kernels durchgesetzt wird.

4.6.2 Einsatz von eBPF-LSM statt klassischem Kernelmodul

Ein klassisches LSM ist Teil des Kernelcodes oder wird als Kernelmodul entwickelt. Änderungen an einem solchen Modul erfordern einen erneuten Build- und Ladeprozess. Für einen experimentellen Prototyp führt dies zu vergleichsweise langen Entwicklungsiterationen.

Gewählt wurde daher die Implementierung als eBPF-LSM. Diese Implementierung erlaubt es, eBPF-Programme an LSM-Hooks auszuführen und damit Zugriffskontrollentscheidungen im Kernel-Ausführungspfad zu treffen [13]. Die Programme können zur Laufzeit geladen werden und werden vor der Ausführung durch den eBPF-Verifier geprüft [15].

Ein weiterer Grund für diese Entscheidung ist die Möglichkeit, eBPF-Programme mit Rust zu entwickeln. Der Prototyp nutzt dafür Aya. Aya ist eine Rust-Bibliothek für die Entwicklung von eBPF-Programmen und stellt damit eine Alternative zu einer C-basierten Implementierung dar [11]. Dadurch können Userspace- und eBPF-Anteile innerhalb desselben Rust-Workspaces entwickelt werden. Zusätzlich können gemeinsame Datenstrukturen und Teile der Policy-Auswertungslogik in einem gemeinsamen Crate abgebildet werden. Dies verbessert die Konsistenz zwischen Userspace und Kernelspace und reduziert doppelte Implementierungen.

Eine Alternative wäre die Entwicklung eines klassischen LSM gewesen. Diese Alternative bietet grundsätzlich größere Freiheiten im Kernelcode, erhöht jedoch Entwicklungs- und Wartungsaufwand. Für den Prototyp ist eBPF-LSM geeigneter, da Änderungen schneller getestet werden können und gleichzeitig ein sicherheitsrelevanter Ausführungspunkt im Kernel genutzt wird.

Konsequenzen dieser Entscheidung sind, dass die Implementierung die Einschränkungen von eBPF beachten muss, insbesondere die Prüfung durch den Verifier, die begrenzte Stack-Nutzung und die eingeschränkte Programmkomplexität. Diese Einschränkungen beeinflussen unter anderem die Aufteilung der eBPF-Programme in verschiedene Tail-Calls und die reduzierte Policy-Repräsentation im Kernelspace.

- 4.6.3 Auswahl der LSM-Hooks**
- 4.6.4 Policy-Verarbeitung im Userspace**
- 4.6.5 Reduzierte Policy-Repräsentation im Kernelspace**
- 4.6.6 Kommunikation über eBPF-Maps**
- 4.6.7 Generationenmodell für Policy-Aktualisierungen**
- 4.6.8 Trennung statischer Policies und Streampolicies**
- 4.6.9 Aufteilung der Policy-Maps nach Hook und Policy-Typ**
- 4.6.10 Tail-Call-basierte Aufteilung der eBPF-Programme**
- 4.6.11 Combining-Strategie für Policy-Entscheidungen**
- 4.6.12 Identifikation von Dateien über Device und Inode**
- 4.6.13 Nachträgliche Überwachung durch den Monitor**
- 4.6.14 Enforcement durch Schließen von File Descriptors**
- 4.6.15 Administration und Diagnose über ein separates Tool**

5 Entwurf und Umsetzung

5.1 Komponente 1

5.2 Komponente 2

5.3 Komponente 3

5.4 Implementierungsdetails

6 Evaluation

6.1 Testumgebung

6.2 Testszenarien

6.3 Ergebnisse

7 Zusammenfassung und Ausblick

Literatur

- [1] James P. Anderson. Computer security technology planning study. Technical Report ESD-TR-73-51, Air Force Electronic Systems Division, 1972. URL <http://csrc.nist.gov/publications/history/ande72.pdf>.
- [2] eBPF Docs. Pinning. <https://docs.ebpf.io/linux/concepts/pinning/>, 2026. Zugriff am 12.06.2026.
- [3] Antonios Gouglidis, Christos Grompanopoulos, and Anastasia Mavridou. Formal verification of usage control models: A case study of usecon using tla+. *Electronic Proceedings in Theoretical Computer Science*, 272:52–64, 2018. doi: 10.4204/EP TCS.272.5. URL <https://arxiv.org/abs/1806.09848>.
- [4] Vincent C. Hu, David Ferraiolo, Rick Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone. Guide to attribute based access control (abac) definition and considerations. Technical Report NIST Special Publication 800-162, National Institute of Standards and Technology, 2014. URL <https://csrc.nist.gov/pubs/sp/800/162/upd2/final>.
- [5] Michael Kerrisk. bpf(2) linux manual page. <https://man7.org/linux/man-pages/man2/bpf.2.html>, 2026. Zugriff am 12.06.2026.
- [6] Michael Kerrisk. open(2) linux manual page. <https://man7.org/linux/man-pages/man2/open.2.html>, 2026. Zugriff am 12.06.2026.
- [7] Michael Kerrisk. proc_pid_fd(5) linux manual page. https://man7.org/linux/man-pages/man5/proc_pid_fd.5.html, 2026. Zugriff am 12.06.2026.
- [8] OASIS. extensible access control markup language (xacml) version 3.0. <https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-en.html>, 2010. Zugriff am 12.06.2026.
- [9] Rusty Russell. Unreliable guide to hacking the linux kernel. <https://docs.kernel.org/kernel-hacking/hacking.html>, 2026. Zugriff am 12.06.2026.
- [10] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. <https://web.mit.edu/Saltzer/www/publications/protection/>, 1975. Online-Fassung, Zugriff am 12.06.2026.
- [11] The Aya Contributors. Building ebpf programs with aya. <https://aya-rs.dev/book/>, 2026. Zugriff am 12.06.2026.
- [12] The Kernel Development Community. Bpf design q&a. https://docs.kernel.org/bpf/bpf_design_QA.html, 2026. Zugriff am 12.06.2026.
- [13] The Kernel Development Community. Lsm bpf programs. https://docs.kernel.org/bpf/prog_lsm.html, 2026. Zugriff am 12.06.2026.
- [14] The Kernel Development Community. Bpf maps. <https://docs.kernel.org/bpf/maps.html>, 2026. Zugriff am 12.06.2026.

- [15] The Kernel Development Community. eBPF verifier. <https://docs.kernel.org/bpf/verifier.html>, 2026. Zugriff am 12.06.2026.
- [16] The Kernel Development Community. Linux security module usage. <https://docs.kernel.org/admin-guide/LSM/index.html>, 2026. Zugriff am 12.06.2026.
- [17] Chris Wright, Crispin Cowan, Stephen Smalley, James Morris, and Greg Kroah-Hartman. Linux security modules: General security support for the linux kernel. In *11th USENIX security symposium (USENIX Security 02)*, 2002.